

AR Workbench — Developer Documentation

Sep 23, 2026 · @Someone

Purpose and audience

This is the developer-facing reference for the Denial & AR Workbench codebase — how it's built, not what it's for (see the separate AR Workbench Build Reference for functionality mapping aimed at stakeholders). It's meant for an engineer picking up this codebase for the first time: where things live, the patterns the app is built on, the exact business-rule logic behind derived fields, and how changes get tested, bundled, and shipped.

The codebase has no build step and no framework — it's ~4,900 lines of vanilla JavaScript across 14 files, loaded as plain `<script>` tags over a static JSON dataset. It ships two ways from the same source tree: as a live Claude Artifact and as a single self-contained HTML file (`LRN_Denial_AR_Workbench.html`) that runs offline with no server. Every change is expected to update both.

Application architecture

Everything hangs off one global, `window.App`. There's no module system — every file appends to the same object, so load order is load-bearing:

```
data.js → core.js → actions.js → charts.js → dashboard.js → workqueue.js →  
claimdetail.js → assignment.js → mywork.js → qaapproval.js → analytics.js →  
admin.js → client.js → main.js
```

`data.js` must load first (it defines `App.data`); `core.js` second (it defines nearly every shared helper — formatting, permissions, table/modal components, business-rule derivations — that every later file calls into). A new file gets appended after `client.js` and before `main.js`, and its `<script>` tag added to `index.html` in the same position.

Three conventions organize the code within that flat namespace:

- **`App.views.<routeId>`** — one render function per navigation route (`App.views.dashboard`, `App.views.workqueue`, `App.views.assignment`, etc.), each taking the DOM container to render into and reading

`App.state / App.scopedClaims()` for its data. `App.renderRoute()` (`core.js`) looks up the current route in `App.NAV`, checks the signed-in role against it, and calls the matching `App.views.*` function.

- **App.actions.<name>** — every state-mutating operation (`assignClaim`, `bulkAssign`, `qaDecision`, `escalateClaim`, `resolveAgentRequest`, etc., all in `actions.js`). Actions are the only code allowed to call `App.saveOverride`; views never mutate claims directly.
- **Shared components** (`core.js`) — `App.createTable`, `App.multiSelectField / wireMultiSelect`, `App.openModal / closeModal`, `App.confirmAction`, `App.toast` — reused across every view rather than re-implemented per screen.

`main.js` is the entry point: it wires the login/logout flow and the `app:data-ready` event (fired once `data.js` finishes fetching `data/claims.json` and `data/meta.json`) to the first render.

Global state, roles, and permissions

`App.state` (`core.js`) is the single mutable state object the whole UI reads and writes:

```
App.state = {
  user: null,           // { username, role, agentCode?, clientLock?, access? }
}
route: 'dashboard',
routeParams: {},
clientScope: 'All Clients',
tableStates: {},      // per-view table UI state, keyed by view id
};
```

Six roles are declared once in `App.ROLE_META` (`admin`, `manager`, `lead`, `agent`, `qa`, `viewer`) with their display labels, and once in `App.PERMS` with a fixed permission matrix per role: `assign`, `editClaim`, `qaDecide`, `approve`, `manageUsers`, `viewAudit`, `manageSettings`, `allClients`, `viewClientMgmt`. `App.can(perm)` is the single check every view and action uses — `!!(App.state.user && App.PERMS[App.state.user.role] && App.PERMS[App.state.user.role][perm])` — never a direct role-name comparison, so a permission change is a one-line edit to `App.PERMS`, not a hunt through every view.

Navigation is data, not code: `App.NAV` is an array of `{ id, label, icon, roles: [...], count? }` entries, one per route. `App.renderNav()` filters this array by the signed-in role to build the sidebar, and `App.renderRoute()` checks the same array before rendering a route, so a role's visible navigation and its enforced access can never drift apart — there's exactly one list to edit when adding a route or changing who can see it. The optional `count` key names a field on `App.counts()` (`core.js`) used to badge the nav item (e.g. `unassigned`, `qaqueue`, `overdue`).

Access-scoping model

A user's data scope is normalized to one shape by `App.resolveAccess(u)`:

```
App.resolveAccess = function (u) {
  if (!u) return { level: 'all' };
  if (u.access && u.access.level) return u.access;
  if (u.clientLock) return { level: 'client', client: u.clientLock }; //
  legacy fallback
  return { level: 'all' };
};
```

`{level}` is one of `'all' | 'client' | 'clinic' | 'provider'`, with `client`, `clinic`, or `provider` set as appropriate. Every other access helper builds on this normalized shape, so a future access level only needs a case added here and in `App.scopedClaims`:

- **`App.allowedClients()`** — the client name(s) a user can see: `[access.client]` if scoped, else every client in `App.data.meta.clients`.
- **`App.scopedClaims()`** — the single function every view calls for its data. Filters `App.data.claims` by `allowedClients()`, then narrows by `orderingPractice` (clinic level) or `renderingProvider` (provider level), then by `agentCode` for the `agent` role, then by the global `clientScope` dropdown state.
- **`App.getClinicsForClient(client)` / `App.getProvidersForClient(client)`** — sorted, deduplicated `orderingPractice` / `renderingProvider` values pulled live from `App.data.claims` for a given client, used to populate the cascading Access Level → Client → Clinic/Provider picker in User Management (`js/admin.js`, `App.views.users`) so an admin can only grant access to real entities, never free text.
- **`App.accessLabel(access)`** — the human-readable summary shown in the Users table and the login roster (`"Client → Clinic (Clinic)"`, etc.).

`App.login(username)` looks a username up in `App.PERSONAS` (the seeded demo roster in `core.js`) or `App.loadLocal('lrn_extra_users', [])` (users created via User Management this session), so a newly granted access level is immediately testable by signing in as that user — not just a cosmetic table row.

Shared UI components

All defined in `core.js` and reused by every view rather than reimplemented per screen:

Component	Signature	What it does
<code>App.createTable</code>	<code>(container, opts)</code>	The workhorse table component: sortable columns (<code>opts.columns[].sortValue</code>), pagination (<code>opts.pageSize</code>), CSV export, column visibility toggling, and optional row selection. <code>opts.selectable</code> turns on bulk-select checkboxes; <code>opts.rowSelectable(row)</code> gates which rows get one — a row failing the predicate renders no checkbox and is excluded from "select all" (built for "can't approve your own QA work" / "can't re-resolve an already-resolved request," reused wherever a bulk action has row-level eligibility rules). <code>opts.toolbarExtra</code> injects a custom button (e.g. "Approve Selected"); <code>opts.onSelectionChange</code> fires as selection changes.

Component	Signature	What it does
<code>App.multiSelectField / App.wireMultiSelect</code>	<code>(id, label, options, selected, onChange)</code>	The standard filter-dropdown popover used on every filter bar in the app — renders the chip/trigger, <code>wireMultiSelect</code> wires its popover's checkbox behavior and calls back with the new selection.
<code>App.openModal / App.closeModal</code>	<code>({ title, bodyHTML, wide, footButtons, onMount, closeOnBackdrop })</code>	The modal shell. <code>onMount(backdrop)</code> runs once the modal is in the DOM — used to wire dynamic behavior inside it, like the cascading Access Level picker or a bulk-action confirmation summary.
<code>App.confirmAction</code>	<code>(message, onConfirm, opts)</code>	The confirm-before-destructive-action dialog shown ahead of every bulk or irreversible operation (bulk reassign, bulk approve, bulk resolve).
<code>App.toast</code>	<code>(msg, type, ms)</code>	Transient notification banner; actions pass <code>{ silent: true }</code> and skip their own toast when they're being called in a loop by a bulk action, which then shows one summary toast instead of one per row.
<code>App.charts.*</code>	<code>barList, comparisonBars, donut, paymentBar, deltaChip, sequentialRamp</code>	Small dependency-free chart primitives (charts.js) used across Dashboard and Analytics — no charting library.

A new bulk-action screen should follow the same shape as QA Verification and Escalation & Reassignment Requests: `App.createTable` with `selectable` + `rowSelectable`, a `toolbarExtra` button, a corresponding `App.actions.bulk*` function that loops the

single-item action with `{ silent: true }` and tallies a result summary, and `App.confirmAction` before applying it.

Persistence model

There is no backend. `js/data.js` boots the app: it fetches `data/claims.json` and `data/meta.json`, merges in any locally-saved claim overrides, and fires an `app:data-ready` event once `App.data = { claims, meta, loaded: true }` is populated.

```
App.saveOverride = function (claimId, patchFn) {
  // patchFn(claim) mutates the in-memory claim; we also persist the *whole*
  // claim object (simplest reliable "simulated persistence" for a client-
  // only demo).
  const claim = App.data.claims.find(c => c.claimId === claimId);
  if (!claim) return;
  patchFn(claim);
  const store = loadOverrides(); // reads localStorage['lrn_ar_workbench_v1']
  store[claimId] = claim;
  localStorage.setItem('lrn_ar_workbench_v1', JSON.stringify(store));
};
```

Every `App.actions.*` function that changes a claim goes through `App.saveOverride`, never a direct mutation — that's what makes a change durable across a page reload within the same browser. On boot, any claim with a saved override entirely replaces the matching seed claim.

Admin-editable master lists (denial categories, panel types, non-collectible codes, root causes, fix/resolution options, etc.) and session-created users use the same pattern at a coarser grain: `App.loadLocal(key, fallback)` / `App.saveLocal(key, val)`, a generic `localStorage` get/set with JSON serialization and a safe fallback if storage is unavailable.

This is a demo-grade data layer, not a production persistence model: single-browser, single-tab, no server, no conflict resolution, and `App.resetDemoData()` simply clears the `localStorage` key and reloads. It's intentionally the simplest thing that makes every workflow genuinely interactive rather than mocked.

Module & Data Reference

Module reference

Load order left to right. Line counts are approximate, as of the current build.

File	Lines	Responsibility	Key exports
<code>data.js</code>	60	Boots the app: fetches the dataset, merges saved overrides, exposes <code>App.data</code>	<code>App.saveOverride</code> , <code>App.resetDemoData</code>
<code>core.js</code>	1,154	Foundation: icons, role/permission metadata, formatters, access-scoping, claim-lifecycle and AR-queue derivations, <code>App.createTable / openModal / confirmAction / multiSelectField</code> , routing, login/logout	<code>App.ROLE_META</code> , <code>App.PERSONAS</code> , <code>App.NAV</code> , <code>App.PERMS</code> , <code>App.can</code> , <code>App.fmt</code> , <code>App.resolveAccess</code> , <code>App.scopedClaims</code> , <code>App.classifyQueue</code> , <code>App.isFinanciallyClosed / isWorkComplete / isOpenInsuranceArClaim</code> , <code>App.daysSinceLastTouch</code> , <code>App.createTable</code> , <code>App.openModal</code> , <code>App.navigate</code> , <code>App.login / logout</code>
<code>actions.js</code>	386	Every state-mutating operation, each ending in <code>App.saveOverride</code>	<code>App.actions.assignClaim / bulkAssign</code> , <code>requestReassignment</code> , <code>resolveAgentRequest / bulkResolveAgentRequests</code> , <code>setStatus</code> , <code>logFollowUp</code> , <code>escalateClaim</code> , <code>approveCipCase / rejectCipCaseToAgent</code> , <code>submitCipClientResponse / reviewCipClientResponse</code> , <code>qaDecision / bulkQaApprove</code> , <code>processAutoAdjustment</code>
<code>charts.js</code>	99	Dependency-free chart primitives used by Dashboard and Analytics	<code>App.charts.barList</code> , <code>comparisonBars</code> , <code>donut</code> , <code>paymentBar</code> , <code>deltaChip</code> , <code>sequentialRamp</code>
<code>dashboard.js</code>	176	Dashboard KPI/overview route	<code>App.views.dashboard</code>
<code>workqueue.js</code>	255	Master claim list, filtering, saved views, the shared assign modal	<code>App.views.workqueue</code> , <code>App.loadSavedViews / persistSavedViews</code> , <code>App.openAssignModal</code>
<code>claimdetail.js</code>	409	Single-claim detail panel: overview, CPT lines, denial info, follow-ups, activity log, QA tab, CIP cases, the follow-up modal	<code>App.openFollowUpModal</code>
<code>assignment.js</code>	241	Assignment Management: Unassigned Claims, Create Assignment Batch, Assigned Claims — Bulk Reassign	<code>App.views.assignment</code>
<code>mywork.js</code>	324	An agent's personal caseload (My Work) and Follow-Up Management	<code>App.views.mywork</code> , <code>App.views.followup</code>
<code>qaapproval.js</code>	141	QA Verification queue, including bulk approve	<code>App.views.qa</code>
<code>analytics.js</code>	297	Analytics charts and the Reports view	<code>App.views.analytics</code> , <code>App.views.reports</code> , <code>App.buildArProgressSummary</code>

File	Lines	Responsibility	Key exports
<code>admin.js</code>	1,133	Client Management, Escalation & Reassignment Requests (incl. bulk resolve), CIP approval modals, User Management, Audit Logs, System Settings / Master File Maintenance	<code>App.views.clients / users / audit / settings</code> , <code>App.openBulkCipApproveModal / openBulkCipRejectModal</code> , <code>App.loadLocal / saveLocal</code>
<code>client.js</code>	221	Client/Viewer-facing Escalation Requests (Client CIP)	<code>App.views['client-cip']</code>
<code>main.js</code>	15	Entry point — wires <code>app:data-ready</code> and login/logout to the first render	—

A new feature typically touches three layers: a business-rule helper in `core.js` (if it needs a new derived field or classification), one or more `App.actions.*` functions in `actions.js`, and the view file for the screen it appears on.

claims.json schema

`data/claims.json` is a flat array of 227 claim objects (minified, ASCII-safe JSON). Every claim carries these fields:

Group	Fields
Identification	<code>claimId</code> , <code>patientAcct</code> , <code>client</code> , <code>payer</code> , <code>secondaryPayer</code> , <code>financialClass</code> , <code>panel</code>
Providers & service	<code>billingProvider</code> , <code>renderingProvider</code> (referring provider), <code>orderingPractice</code> (clinic), <code>dos</code> , <code>dosEnd</code> , <code>cptLines</code>
Claim status	<code>claimStatus</code> , <code>priority</code>
Financials	<code>billedAmount</code> , <code>allowedAmount</code> , <code>insurancePayment</code> , <code>patientPayment</code> , <code>adjustmentAmount</code> , <code>insuranceBalance</code> , <code>patientBalance</code> , <code>totalBalance</code>
Denial detail	<code>denialCode</code> , <code>denialCategory</code> , <code>denialReason</code> , <code>denialDate</code> , <code>denialRootCause</code>

Group	Fields
Workflow state	<code>workflowCategory</code> , <code>workflowStatus</code> , <code>assignedAgent</code> , <code>assignedAgentName</code> , <code>submissionDate</code> , <code>lastBillDate</code> , <code>paymentDate</code> , <code>agingDays</code> , <code>agingBucket</code> , <code>tflRisk</code> , <code>tflDeadline</code> , <code>lastFollowUp</code> , <code>nextFollowUp</code> , <code>workedStatus</code>
Recovery & AR remediation	<code>expectedPayment</code> , <code>actualPayment</code> , <code>paymentVariance</code> , <code>paymentPct</code> , <code>underpaymentAmount</code> , <code>appealRequired</code> , <code>recoveryStatus</code> , <code>recoveredAmount</code> , <code>remainingAR</code> , <code>initialInsuranceAR</code> , <code>arCategory</code> , <code>arRemark</code> , <code>fixResolution</code>
Nested records	<code>activity[]</code> (audit log — every claim has at least a "Claim Identified" ETL entry), <code>followUps[]</code> , <code>qa{}</code> (QA review state), <code>agentRequests[]</code> (escalation/reassignment requests raised by an agent), <code>cipCases[]</code> (Corrective Implementation Plan cases)
Derived/flag fields	<code>escalationApproved</code> (set once QA approves a linked escalation), <code>adHocFollowUpAssigned</code> (set by <code>assignClaim</code> when a closed/zero-balance claim is hand-assigned anyway)

Most fields on this list are source data; the derived business-state fields (financial closure, staleness, AR-queue bucket, follow-up actionability) are never stored — they're computed on read by the `core.js` helpers documented in the Business Logic Reference tab, from the raw fields above.

meta.json reference data

`data/meta.json` holds every piece of reference data the app draws its dropdowns, master lists, and business-rule tables from. Its top-level keys:

Key	Shape	Used for
<code>dataRefresh</code>	<code>{ refreshDate, sourcePeriodStart, sourcePeriodEnd, note }</code>	Dashboard "as of" framing text
<code>clients</code>	<code>[{ id, name }]</code>	3 seeded clients (Summit Toxicology Labs, Highland Diagnostic Labs, Cascade Precision Labs)
<code>payers</code>	<code>string[]</code>	8 seeded payers
<code>panelTypes</code>	<code>string[]</code>	Panel Type filter/analytics dimension (admin-editable via Master File Maintenance)
<code>agents</code>	<code>[{ code, name }]</code>	10 seeded agents; <code>code</code> matches <code>claim.assignedAgent</code>
<code>qaReviewers</code>	<code>string[]</code>	QA reviewer roster
<code>denialCategories</code>	<code>string[]</code>	Admin-editable master list
<code>workflowStatuses</code>	<code>string[]</code>	<code>['Unassigned', 'Assigned', 'Submitted for QA', 'QA Rejected', 'Completed']</code> — the full <code>workflowStatus</code> state machine
<code>agingBuckets</code>	<code>string[]</code>	6 buckets, 0-30 through 181+ days
<code>financialClasses</code>	<code>string[]</code>	Commercial / Medicare / Medicaid / Self Pay
<code>tf1ThresholdDays</code>	<code>{ [financialClass]: days }</code>	Timely-filing-limit thresholds per financial class
<code>workflowTemplates</code>	<code>{ [key]: { label, stages[] } }</code>	5 named workflow templates (doc_required, non_responded, coding_edit, partial_payment, denied), each an ordered stage list
<code>demoUsers</code>	<code>[{ username, role, clients }]</code>	Legacy seed list superseded by <code>App.PERSONAS</code> in <code>core.js</code> (which adds the <code>access grant</code> shape)
<code>commentsFramework</code>	<code>{ claimTypes, followUpTypes, claimStatuses, denialRootCauses, fixResolutions, fixResolutionsByStatus, cip }</code>	The full vocabulary behind follow-up logging, denial root cause, and fix/resolution selection, including which fix/resolution options are valid for a given claim status, and the CIP category/required-info lists

Key	Shape	Used for
<code>revenueConfidenceTiers</code>	<code>number[]</code>	<code>[1.0, 0.5, 0.3]</code> — confidence weighting tiers
<code>arQueues</code>	<code>[{ id, label, priority, sub: [...] }]</code>	The full AR queue taxonomy <code>App.classifyQueue</code> sorts claims into — 12 top-level queues (Closed, Non-Responded AR, Denied, Partially Paid, Patient AR, Partially Adjusted, Client Escalations Pending, CIP Response Received, Re-follow-up Required, Submitted for QA, Completed, QA Rejected), several with sub-queues splitting collectible vs. non-collectible
<code>nonCollectibleDenialCodes</code>	<code>string[]</code>	Denial codes (<code>0A257</code> , <code>PR288</code> , <code>PR27</code> , <code>PR31</code> seeded) that mark a claim non-collectible in <code>arqIsNonCollectible</code>

Every list here is overridable per-session via `App.loadLocal / saveLocal` — Master File Maintenance (Settings, admin.js) edits `localStorage` copies layered on top of these seed defaults, never the file itself.

Business Logic Reference

Financial closure and claim-lifecycle helpers

All three live in `core.js` and are the vocabulary every other view/rule is built from. None of them are stored fields — each is computed on read from the claim's live financial/status fields, so they can never drift out of sync with the underlying data:

```
// A claim is financially closed once neither the insurer nor the patient
// owes
// anything further.
App.isFinanciallyClosed = (c) => c.remainingAR <= 0.005 && c.patientBalance
<= 0.005;

// An AR agent's work on a claim is "done" once it's either financially
// closed
// or QA has approved the latest submission (workflowStatus === 'Completed').
App.isWorkComplete = (c) => App.isFinanciallyClosed(c) || c.workflowStatus
=== 'Completed';

// Every agent-facing / operational view (Work Queue, Dashboard's Claim Queue
// Volumes, Assignment Management, Follow-Up Management) is workable-
// insurance-AR
// inventory: it should only ever surface a claim that still carries an open
// insurance balance right now. Deliberately a direct field check rather than
// a
// check on the AR-queue topId/priority, because a claim can be pushed to $0
// insurance balance at any point – including mid-QA-cycle – and
// workflowStatus
// doesn't always get formally closed out the same moment.
App.isOpenInsuranceArClaim = function (claim) {
  return claim.insuranceBalance > 0.005;
};
```

`0.005` is the epsilon used everywhere balances are compared to zero, to absorb floating-point rounding noise in the seeded financial data — use it (or a matching helper) rather than a bare `<= 0` or `> 0` comparison anywhere a new rule touches a balance field.

`isWorkComplete` and `isOpenInsuranceArClaim` are NOT opposites, and that gap is intentional and load-bearing: a claim can be `workflowStatus === 'Completed'` (work-complete) while `insuranceBalance > 0` (still open) — a QA-approved follow-up whose

payer hasn't paid out yet. Assignment Management's Bulk Reassign table deliberately includes these claims (see the Assignment Management rules section), while the Agent Workload chart's "open claims" count deliberately excludes them — pick the right one of the two checks per new feature rather than assuming they're interchangeable.

Staleness and follow-up actionability

Two related but distinct "how stale is this" rules, each solving a different problem — easy to conflate, and the wrong one has previously caused a real bug (see the callout below).

App.daysSinceLastTouch(claim) — days since the most recent entry in the claim's own activity[] log, not since lastFollowUp:

```
App.daysSinceLastTouch = function (claim) {
  const acts = claim.activity || [];
  if (!acts.length) return null;
  const latest = acts.reduce((max, a) => (a.timestamp > max ? a.timestamp :
max), acts[0].timestamp);
  const d = new Date(latest.slice(0, 10) + 'T00:00:00');
  if (isNaN(d)) return null;
  return Math.round((App.TODAY - d) / 86400000);
};
```

This exists because `claim.lastFollowUp` is unconditionally `null` for any claim that has never been assigned — it's only ever set by `App.actions.logFollowUp`. A naive reuse of the follow-up-due logic below would treat every never-touched unassigned claim as "not stale" (a null short-circuits to false). `daysSinceLastTouch` instead reads the activity log, which every claim has at least one entry in (the ETL's "Claim Identified" event), so it correctly measures staleness for both touched and never-touched claims. This is what powers Assignment Management's Unassigned Claims "Days Untouched" column and its 45-day threshold.

App.isFollowUpActionable(claim) — whether a claim belongs in the Follow-Up Management to-do queue, a *different* 45-day check that only applies once a claim has already been worked at least once:

```
App.FU_ACTIONABLE_STATUSES = ['Assigned', 'QA Rejected'];
App.isFollowUpActionable = function (claim) {
  return (App.isOpenInsuranceArClaim(claim) || claim.adHocFollowUpAssigned)
    && App.FU_ACTIONABLE_STATUSES.includes(claim.workflowStatus);
};
```

`arqRefollowupDue(claim)` (also `core.js`, not exported on `App`) is the closely related check used inside `classifyQueue` for a claim already marked `Completed`: it fires once 45+ days have passed since `lastFollowUp`, or `nextFollowUp` has already lapsed. Both `isFollowUpActionable` and `arqRefollowupDue` require `lastFollowUp / nextFollowUp` to be meaningful, i.e. the claim has been worked before — they are not substitutes for `daysSinceLastTouch` on a never-assigned claim.

`App.counts()` and `App.views.followup` (`mywork.js`) both call `App.isFollowUpActionable` directly, rather than each re-implementing the filter, so the Follow-Up nav badge count and the queue's actual contents can't drift apart.

AR queue classification

`App.classifyQueue(claim)` sorts every claim into exactly one leaf of the taxonomy defined in `meta.arQueues` (12 top-level queues, several with sub-queues — see the Module & Data Reference tab). It's a priority-ordered chain of checks, each returning as soon as it matches:

```
App.classifyQueue = function (claim) {
  if (claim.qa && claim.qa.status === 'Awaiting QA') return
    arqLeaf('submittedqa', null);
  if (claim.workflowStatus === 'QA Rejected') return arqLeaf('qarejected',
    null);
  if (claim.workflowStatus === 'Completed') {
    if (claim.remainingAR <= 0.005) return arqFinancialState(claim);
    if (arqHasActiveCip(claim)) return arqLeaf('escalation', null);
    if (arqRefollowupDue(claim)) return arqLeaf('refollowup', null);
    return arqLeaf('completed', null);
  }
  if (arqHasReturnedCipResponse(claim)) return arqLeaf('cipresponse', null);
  return arqFinancialState(claim);
};
```

The ordering matters and encodes real priority: a claim awaiting QA or already QA-rejected is classified by that state regardless of its financials; a `Completed` claim with a still-open CIP case is held in the Client Escalations Pending queue rather than masked by a stale re-follow-up date carried over from before the escalation; a returned CIP client response is surfaced as its own queue before it blends into ordinary raw inventory.

`arqFinancialState(claim)` is the fallback bucket for raw (not mid-QA, not escalated) inventory, keyed off `remainingAR`, `patientBalance`, `recoveredAmount`, and `adjustmentAmount`:

```
function arqFinancialState(claim) {
  const ar = claim.remainingAR, pb = claim.patientBalance, rec =
  claim.recoveredAmount, adj = claim.adjustmentAmount;
  if (ar <= 0.005) {
    if (pb > 0.005) return arqLeaf('patientar', null);
    if (rec > 0.005) return arqLeaf('closed', 'closed_paid');
    return arqLeaf('closed', 'closed_adjusted');
  }
  if (rec <= 0.005 && adj <= 0.005 && claim.denialCategory === 'Unresponsive
  Payers') return arqLeaf('nonresponded', 'nonresponded_collectible');
  if (rec <= 0.005 && adj > 0.005) return arqLeaf('partialadj',
  'partialadj_collectible');
  if (rec > 0.005) return arqLeaf('partial', arqIsNonCollectible(claim) ?
  'partial_noncollectible' : 'partial_collectible');
  return arqLeaf('denied', arqIsNonCollectible(claim) ?
  'denied_noncollectible' : 'denied_collectible');
}
```

`arqIsNonCollectible(claim)` checks the claim's `denialCode` (comma-split, since a claim can carry more than one) against `App.getNonCollectibleCodes()` — the admin-editable Non-Collectible Denial Codes master list. `App.arQueueDisplay(claim)` and `App.arQueueBadgeClass(topId)` render the classification result consistently everywhere a queue label or badge color appears. Adding a new queue means adding its entry to `meta.arQueues` and a matching branch here — never inferring a queue from a UI-layer filter.

Assignment Management business rules

`App.views.assignment` (`assignment.js`) draws three different populations from the same `App.scopedClaims()` rows — deliberately, not by accident — and each has its own code comment explaining why:

```
const UNASSIGNED_UNTOUCHED_DAYS = 45;

// Create Assignment Batch draws from the full unassigned, open-balance pool
// (a claim with no open insurance balance has nothing left for an agent to
// chase) &mdash; batching isn't restricted to stale claims, since
// proactively
// assigning fresh denials the day they're identified is a normal workflow.
const unassignedOpen = rows.filter(c => c.workflowStatus === 'Unassigned' &&
App.isOpenInsuranceArClaim(c));

// The Unassigned Claims table is a narrower, "this needs attention now" view
// of that same pool: still nobody's, still owed, AND untouched for
// UNASSIGNED_UNTOUCHED_DAYS+ days.
const unassigned = unassignedOpen.filter(c => {
  const days = App.daysSinceLastTouch(c);
  return days === null || days >= UNASSIGNED_UNTOUCHED_DAYS;
});

// Agent Workload: "open claims" is live insurance-AR work sitting with an
// agent right now &mdash; not yet work-complete AND still carrying an open
// insurance balance.
const workload = meta.agents.map(a => ({ code: a.code, name: a.name,
  open: rows.filter(c => c.assignedAgent === a.code && !App.isWorkComplete(c)
&& App.isOpenInsuranceArClaim(c)).length }));

// Assigned Claims – Bulk Reassign: every claim with an assigned agent and an
// outstanding insurance balance, including ones marked Completed.
const openAssigned = rows.filter(c => c.assignedAgent &&
App.isOpenInsuranceArClaim(c));
```

Three populations, three different rules, on purpose:

Population	Filter	Why
<code>unassignedOpen</code> → Create Assignment Batch	Unassigned + open balance	No staleness gate — batching fresh denials the day they land is normal
<code>unassigned</code> → Unassigned Claims table	<code>unassignedOpen</code> + untouched 45+ days	The neglect-surfacing view; excludes claims still within normal turnaround
<code>workload.open</code> → Agent Workload chart	<code>!isWorkComplete</code> && <code>isOpenInsuranceArClaim</code>	Counts only genuinely active work, excludes work-complete-but-still- open claims
<code>openAssigned</code> → Bulk Reassign table	<code>isOpenInsuranceArClaim</code> only (no <code>isWorkComplete</code> check)	Deliberately broader than the chart above — includes <code>Completed</code> claims still carrying a balance, since they're still real, reassignable workload even though they're not "active" by the chart's stricter definition

The gap between the Agent Workload chart's count and the Bulk Reassign table's count for the same agent is intentional, not a bug: the chart answers "how much is this agent actively working," the table answers "what could I hand to someone else." Anyone extending this screen should reuse `unassignedOpen` / `unassigned` / `openAssigned` rather than re-deriving a fourth, subtly different filter.

Testing, Build & Release

Testing conventions

Every feature is verified end to end with Playwright browser automation, launched headless against the pre-installed Chromium binary. There are 39 `verify_*.py` scripts in the scratchpad root, one (or a small family) per feature area — for example `verify_access_levels.py`, `verify_qa_bulk_approve.py`, `verify_agent_requests_bulk_resolve.py`, `verify_assignment_untouched_45d.py`, `verify_bulk_reassign.py`, `verify_cip_lifecycle.py`, `verify_master_file_maintenance.py`. Naming convention: `verify_<feature>.py` for the source-tree test, `verify_bundle_<feature>_smoke.py` for the matching smoke test against the built standalone bundle.

Each script follows the same shape: launch Chromium (`p.chromium.launch(executable_path='/opt/pw-browsers/chromium')`), navigate to the app, log in as the persona the scenario needs (`switch_to(page, 'Persona Name')`), drive the UI, and assert against **ground truth computed independently in the browser via `page.evaluate`** — re-deriving the expected result from `App.data.claims` directly (sometimes even re-implementing the business rule under test from scratch, rather than just re-calling the same helper being tested) so a test can't tautologically pass by sharing a bug with the code it's checking. Every check is tallied (`PASS / FAIL` printed per assertion) and the script exits non-zero on any failure or unexpected console/page error.

Local dev workflow: source-tree tests run against a local static server — `nohup python3 -m http.server 8941 &` from the `lrn/` directory — hitting `http://localhost:8941/index.html`. Bundle smoke tests instead load `file:///mnt/user-data/outputs/LRN_Denial_AR_Workbench.html` directly, no server needed. When changing a feature: update the source, restart/confirm the local server is up (`curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8941` should return `200`), run that feature's `verify_*.py` plus any test covering code it touches (a regression pass), then rebuild the bundle and run the matching `verify_bundle_*_smoke.py` before publishing.

One known source of test flakiness, not a real regression: `verify_escalate.py` intermittently times out due to Chromium retrying calls to Google APIs through a restricted sandbox egress proxy, unrelated to app code — a clean retry passes.

Build and release pipeline

The app ships two ways from one source tree, and both are kept current on every change:

1. **Claude Artifact** — a live, versioned, shareable page.
2. **Standalone HTML bundle** — `LRN_Denial_AR_Workbench.html`, a single self-contained file with no server dependency.

`build_bundle.py` (scratchpad root) builds the second from the source tree: it inlines every `js/*.js` file, the stylesheet, and both `data/*.json` files directly into one HTML document, in the same script order `index.html` loads them in. It's re-run after every change, before the bundle smoke tests.

Release checklist for a change:

1. Edit the source files in `lrn/js/` (and `lrn/data/` if the change touches reference data).
2. Run the feature's `verify_*.py` against the local static server, plus a regression pass on any test covering code the change touches.
3. Run `build_bundle.py` to rebuild `LRN_Denial_AR_Workbench.html`.
4. Run the matching `verify_bundle_*_smoke.py` against the rebuilt bundle via `file://`.
5. Publish to the Artifact: `read` the files about to change plus the last known-good published version first, diff them to confirm no drift (nobody else has changed the published artifact since the last publish), then publish the updated files.
6. Deliver the rebuilt bundle to the user.

Step 5's diff-before-publish is the guard against silently overwriting an unrelated change made directly on the published Artifact between sessions — always read before you write when publishing, never assume the last known state is still current.

There is no CI, package manager, or transpiler in this project — no `npm install`, no bundler config, nothing to "build" in the traditional sense beyond `build_bundle.py`'s straight file concatenation. A change is only truly done once both delivery targets (Artifact and bundle) reflect it and their respective tests pass; shipping to only one is treated as an incomplete change, not two separate releases.

CIP Lifecycle & Remaining Modules

claimdetail.js: the Claim Detail view

`App.views['claim-detail']` is the one screen every other queue's row click navigates into (`App.navigate('claim-detail', { claimId, from: <origin route> })`), so every queue passes its own route id as `from` and the Back button reads it back out of `BACK_LABELS` to return the user to where they came from rather than a hardcoded default.

The header renders `App.classifyQueue(claim)` and `App.statusBadgeClass / App.arQueueBadgeClass` (see the Business Logic Reference tab) for its badges, plus any pending `agentRequests[]` entry. A `stepperPhase(status)` lookup (`{Unassigned:0, Assigned:1, 'Submitted for QA':2, 'QA Rejected':2, Completed:3}`) drives the 4-stage visual stepper, and the claim's `workflowCategory` looks up its named stage path from `meta.workflowTemplates`.

Tabs are stored per-claim in `App.state.tableStates['detail_' + claimId + '_tab']` (defaulting to `'overview'`) and rendered conditionally — `qa` only shows once `claim.qa` exists, `cip` only once `claim.cipCases.length`. Each tab has its own render function (`renderOverview`, `renderCpt`, `renderDenial`, `renderFollowUps`, `renderActivity`, `renderQa`, `renderCipCases`), all taking (`panel, claim`).

`renderActions(claim)` gates the header buttons on `App.can('assign')`, `App.can('editClaim')`, and the signed-in role: Assign/Reassign for anyone who can assign; Log Follow-Up Note (only when not `isFinanciallyClosed` and not already `Submitted for QA`); and, only when `App.state.user.role === 'agent'`, Escalate to Supervisor and Request Reassignment. `handleAction(act, claim)` dispatches each button to the matching modal or action call and calls `refreshDetail()` (a thin wrapper on `App.renderRoute()`) afterward to re-render the screen from the updated claim.

Log Follow-Up Note and CIP case creation

`App.openFollowUpModal(claimId, onDone)` (`claimdetail.js`) renders the Comments Framework form from `App.data.meta.commentsFramework` (`claimTypes`, `followUpTypes`, `claimStatuses`, `cip.categories`, `cip.requiredInfo`) plus the admin-editable Denial Root Cause and Fix/Resolution-by-status lists, read through `App.getDenialRootCauses()` / `App.getFixResolutionsByStatus()` so a Master File

Maintenance edit is reflected immediately. Its `onMount` wires the Fix/Resolution options to re-populate whenever Claim Status changes, and toggles the Denial Root Cause field (Denied only) and the CIP fields block (only when Fix/Resolution is "CIP - Client Escalations") accordingly.

Saving calls `App.actions.logFollowUp(claimId, data)`, which unconditionally: appends a `followUps[]` entry; sets `claim.lastFollowUp`, `fixResolution`, `denialRootCause`, `workedStatus = 'Worked'`, `nextFollowUp`; sets `workflowStatus = 'Submitted for QA'` and resets `escalationApproved = false`; and creates a fresh `qa` record (`status: 'Awaiting QA'`, `isEscalation` flagged from the Fix/Resolution). When `data.fixResolution === 'CIP - Client Escalations'`, it additionally pushes a new entry onto `claim.cipCases[]`:

```
if (data.fixResolution === 'CIP - Client Escalations' && data.cip) {
  claim.cipCases = claim.cipCases || [];
  claim.cipCases.push({
    id: newCipCaseId(),
    category: data.cip.category, requiredInfo: data.cip.requiredInfo,
    comment: data.cip.comment,
    followUpDate: today, requestedBy: currentUserLabel(), requestedByRole:
    currentRoleLabel(),
    status: 'Pending Approval', round: 1,
    clientResponse: null, lastReviewDecision: null,
    history: [{ date: nowStamp(), actor: currentUserLabel(), actorRole:
    currentRoleLabel(), action: 'Escalation Logged', note: data.cip.comment }],
  });
}
```

The `followUps[]` entry's own `cip` field keeps the original `{category, requiredInfo, comment}` as a historical record; the `cipCases[]` entry created here is the separate, live, actioned copy that then moves through the approval lifecycle documented next.

CIP case-lifecycle actions

Five actions in `actions.js` drive a `cipCases[]` entry through its `status` state machine (`Pending Approval → Sent to Client → Client Responded → (Sent to Client, round+1) → Returned to Agent`), each guarded by the case's current status so a stale or double-submitted action is a silent no-op rather than a corrupted state:

```

App.actions.approveCipCase = function (claimId, caseId, note, opts = {}) {
  App.saveOverride(claimId, (claim) => {
    const c = findCipCase(claim, caseId);
    if (!c || c.status !== 'Pending Approval') return;
    c.status = 'Sent to Client';
    pushCipHistory(c, { action: 'Approved for Client', note });
    pushActivity(claim, { action: 'CIP Approved for Client', ... });
  });
};

App.actions.rejectCipCaseToAgent = function (claimId, caseId, reason, opts = {}) {
  // guarded on status === 'Pending Approval' – sets status 'Returned to Agent',
  // records lastReviewDecision, then calls reopenClaimForAgent(claim, reason)
};

App.actions.submitCipClientResponse = function (claimId, caseId, { text, attachments }) {
  // guarded on status === 'Sent to Client' – sets clientResponse and status 'Client Responded'
};

App.actions.reviewCipClientResponse = function (claimId, caseId, decision, note, opts = {}) {
  // guarded on status === 'Client Responded'
  // decision 'approved' -> status 'Returned to Agent',
  reopenClaimForAgent(...)
  // decision 'insufficient' -> status back to 'Sent to Client', round += 1
};

```

`reopenClaimForAgent(claim, reason)` (the shared exit path used by both `rejectCipCaseToAgent` and an approved `reviewCipClientResponse`) sets `workflowStatus = 'Assigned'` and `escalationApproved = false`, dropping the claim back into the agent's active worklist so it falls through to its ordinary AR-queue classification and can receive a new follow-up.

Two `core.js` helpers support the CIP queues: `arqHasActiveCip(claim)` (`((claim.cipCases||[]).some(c => c.status !== 'Returned to Agent'))`) is what `classifyQueue` checks to route a `Completed` claim with an open case into the Client Escalations Pending queue ahead of its ordinary financial-state bucket (see the Business

Logic Reference tab); `App.buildCipEntries(rows)` flattens every claim's `cipCases[]` into one row per case (mirroring `App.buildAgentRequestEntries` for agent requests), sorted Client Responded first, then Pending Approval, Sent to Client, Returned to Agent — the shape both the CIP Escalations queue (admin.js) and the client-facing Escalation Requests page (client.js) render from, filtered by status for the client-side view.

Automatic Adjustment and internal escalation

```
// "No touch required": once a denial's code matches the master list, the
// remaining balance is written off automatically rather than routed to
// an agent. Triggered in a batch by a manager/lead/admin from the Work
// Queue.
App.actions.processAutoAdjustment = function (claimId) {
  App.saveOverride(claimId, (claim) => {
    const prevBal = claim.remainingAR;
    claim.adjustmentAmount = +(claim.adjustmentAmount +
    claim.remainingAR).toFixed(2);
    claim.remainingAR = 0;
    pushActivity(claim, { action: 'Automatic Adjustment', ..., user:
    'System', role: 'Automation' });
  });
};
App.actions.bulkProcessAutoAdjustment = function (claimIds) {
  claimIds.forEach(id => App.actions.processAutoAdjustment(id));
};
```

Note the explicit `user: 'System', role: 'Automation'` on the activity entry — every other `pushActivity` call defaults to `currentUserLabel() / currentRoleLabel()` (the signed-in user), so this is the one path in the codebase that writes an activity entry attributable to the system itself rather than a person, and Audit Logs / the Activity Timeline render it as such.

`App.actions.escalateClaim(claimId, note)` is the internal-only counterpart to a CIP escalation: it sets `workflowStatus = 'Submitted for QA'` with `qa.isEscalation = false` (so it never routes to Client Escalations Pending) and separately pushes an `agentRequests[]` entry of `type: 'Escalation to Supervisor'`, which is what surfaces it in the Escalation & Reassignment Requests queue via `App.buildAgentRequestEntries`. It never touches `cipCases[]` — the two escalation paths (this one, and "CIP - Client Escalations" via `logFollowUp`) are entirely separate code paths that happen to share the word "escalation"; do not conflate them when extending either.

The remaining views

App.views['data-processing'] (admin.js) is presentation-only simulation: the "Run Data Processing" button drives a chain of `setTimeout` calls that update a status line through a few fake stages before confirming completion — it never mutates `App.data`, since the dataset already reflects "this cycle's" output by construction. It does call two real `core.js` helpers for the Denial Analysis Report card:

```
App.buildDenialCodeHighlights(rows) and
App.denialHighlightsTableHTML(highlights) /
App.wireDenialHighlightsLinks(container, highlights), which group and rank
denial codes and wire their rows to navigate into the matching claims — the same helpers
used on the Dashboard.
```

App.views.clients (admin.js) reads `App.loadLocal('lrn_client_disabled', {})`, a simple `{[clientName]: boolean}` map toggled by a Deactivate/Reactivate button per client card. It does not filter that client out of `App.scopedClaims()` or anywhere else — it's a status badge only, not an access change, so deactivating a client in this view does not hide its claims from other screens.

App.views.audit (admin.js) builds its row set by flattening every scoped claim's `activity[]` with the owning `claimId / client` attached (`rows.forEach(c => c.activity.forEach(a => entries.push({...a, claimId: c.claimId, client: c.client})))`), sorted newest-first, then filtered by the same `App.multiSelectField` pattern as every other queue. There is no separate audit-log data source — it is purely a projection over the claims' own `activity[]` arrays, so it can never drift from what the Claim Detail screen's Activity Timeline shows for an individual claim.

client.js is the Client/Viewer-role counterpart to the CIP Escalations queue: `App.views['client-cip']` calls the same `App.buildCipEntries` helper, filtered to exclude 'Pending Approval' (the client never sees an unapproved request) and further scoped to their own client/clinic/provider by `App.scopedClaims()`. Beyond the individual `openClientRespondModal` (text + up to `CIP_ATTACH_MAX_FILES` (3) attachments capped at `CIP_ATTACH_MAX_BYTES` (250KB) each, base64-encoded since the whole claim record round-trips through `localStorage`), it supports a bulk path: a CSV response template (columns matching the open cases) can be downloaded via `App.exportCsvModal`, filled in Excel, and re-uploaded; `handleBulkCipUpload(text)` parses it with `App.parseCsv` and calls `App.actions.submitCipClientResponse` once per matched row.

Cross-cutting utility helpers

Three more core.js helpers are used across most `App.createTable` screens but weren't called out in the Architecture tab's component table:

Helper	Signature	What it does
<code>App.loadSavedViews / App.persistSavedViews</code>	<code>(key) / (key, views)</code>	Generic named-filter-set persistence, <code>localStorage</code> key <code>'lrn_views_' + key</code> — each queue passes its own <code>key</code> (e.g. <code>'workqueue'</code>) so saved views don't collide across screens.
<code>App.exportCsvModal</code>	<code>(name, columns, rows)</code>	Writes the currently filtered/sorted rows to a downloadable CSV, using each column's <code>csvValue</code> when present (falling back to its <code>render /raw</code> value) — so CSV export always matches what's on screen, not the full unfiltered dataset.
<code>App.parseCsv</code>	<code>(text)</code>	The matching RFC4180-ish reader, used by the client-facing bulk CIP-response upload (<code>handleBulkCipUpload</code> in <code>client.js</code>) and Master File Maintenance's CSV upload paths (<code>handleMfChipListUpload</code> , <code>handleMfMapUpload</code> , <code>handleMfFixByStatusUpload</code> in <code>admin.js</code>).
<code>App.openColumnMenu</code>	<code>(anchor, columns, hiddenCols, onChange)</code>	The show/hide-columns popover wired into <code>App.createTable</code> 's own toolbar; <code>onChange</code> receives the updated hidden-set and the table re-renders.

All four are called from inside `App.createTable`'s own toolbar rendering except `parseCsv`, which is a standalone reader other upload flows reuse directly. A new table screen gets saved views, CSV export, and column visibility for free simply by using `App.createTable` — there is no need to wire any of the three by hand; they only need attention when a table's columns define `csvValue` for a field whose displayed `render` output wouldn't make sense in a spreadsheet cell (money formatting, badges, multi-line HTML, etc.).